



One-click sovereign bond yield for EVM users, settled on Stellar

Technical Infrastructure Description

SCF Build Application / Integration Track

1. Introduction

High-Level Overview

Prism is a live multichain consumer DeFi hub: swaps, liquidity pools, perpetual futures, prediction markets, yield, and portfolio management in a single interface, currently deployed on MegaETH and Arbitrum. Since launching in March 2026, Prism has onboarded 6,000+ users in its first three months with zero paid acquisition; over that period, monthly volume grew from \$6.95M (March) to \$33.3M (May) and TVL peaked at \$4M. Prism was selected into the Arbitrum Mentorship Program (13 teams from 900+ applicants).

This submission integrates Stellar as the **settlement and issuance rail for Prism's RWA yield module**. The assets are Stellar-native: Etherfuse sovereign stablebonds issued on Stellar, held on Stellar accounts, settled on Stellar. The friction is access: reaching these assets from an EVM wallet takes a multi-step bridge journey, which in practice keeps EVM users out of the ecosystem. The venue is the right one: Stellar is the #4 network for permissionless tokenized RWA value (RWA.xyz), and our first-market issuer's deepest liquidity sits there. Prism's architecture closes the gap by making that venue feel EVM-native: one signature, no seed phrase, no XLM. The award funds the deposit rail and the yield module to mainnet, adds an open-source Soroban vault strategy on DeFindex (section 5.4), and routes an existing EVM user base onto Stellar rails.

A POC of the earn module, with a functional Stellar wallet integration on testnet, is already live at **stellar.prismfi.cc/earn**, built before requesting any funding. This document describes the architecture of the full integration, the design decisions behind it, and how each component maps to the deliverables in the Build submission.

Design principle

Every technical choice in this document was made on a single criterion: **efficiency**. Minimum new surface area, maximum reuse of Prism's production infrastructure and of battle-tested Stellar ecosystem components, shortest verifiable path to mainnet. Where a custom build and an ecosystem building block both solve a problem, the building block wins: twelve ecosystem integrations, zero custom core infrastructure.

2. Definitions, Acronyms and Abbreviations

- **Intent**: A declarative cross-chain order settled by a solver network (here, NEAR Intents).
- **CCTP V2**: Circle's Cross-Chain Transfer Protocol: canonical 1:1 burn-and-mint of native USDC across supported chains. Live on Stellar mainnet since May 2026.
- **Trustline**: An explicit opt-in held by a Stellar account to hold a given asset. Each trustline locks a small XLM reserve.
- **Sponsored reserves**: A Stellar protocol feature letting one account (Prism) pay the base and trustline reserves of another account (the user), removing the need for users to hold XLM.
- **Fee-bump transaction**: A Stellar transaction wrapper letting Prism pay network fees on behalf of a user-signed transaction.
- **Stablebond**: A tokenized sovereign bond instrument issued on Stellar by a regulated issuer (Etherfuse, starting with USTRY) as a standard Stellar asset; an accreting-value token whose redemption value increases with accrued yield.
- **SDEX**: Stellar's native on-ledger decentralized exchange, with order books and path payments built into the protocol.
- **DeFindex**: SCF-funded vault infrastructure on Stellar; apps plug one API to offer vault strategies routing into ecosystem DeFi protocols.
- **Reflector**: Stellar's native oracle network, providing on-chain price and FX feeds.
- **Soroswap**: Stellar's AMM and DEX aggregator; the Aggregator splits orders across SDEX, Soroswap, and Aquarius for best execution.
- **Blend**: An SCF-funded lending protocol on Stellar; idle USDC is supplied to its pool for interim yield.

- **SEP-10:** Stellar's Web Authentication standard: a challenge-response that yields a session token tied to a Stellar account.
- **Stellar RPC:** The JSON-RPC API recommended by SDF as the entry point for new applications, replacing Horizon for most use cases.

3. Architecture Overview

C4 L1: System Context

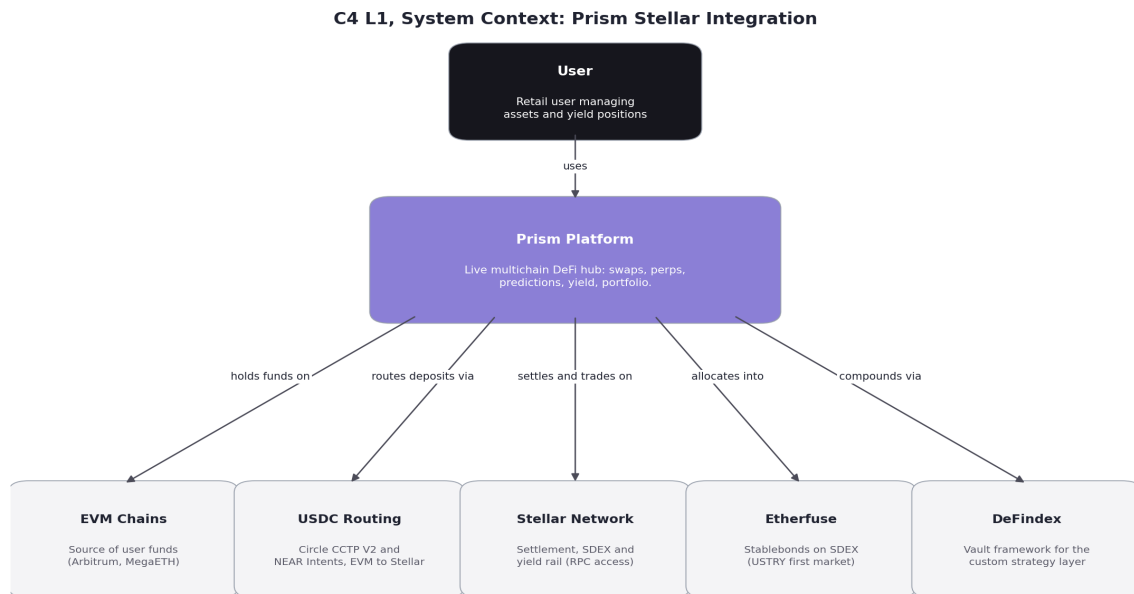


Figure 1: System context. Prism orchestrates the external Stellar ecosystem; Stellar is the settlement, trading and yield rail.

C4 L2: Container View

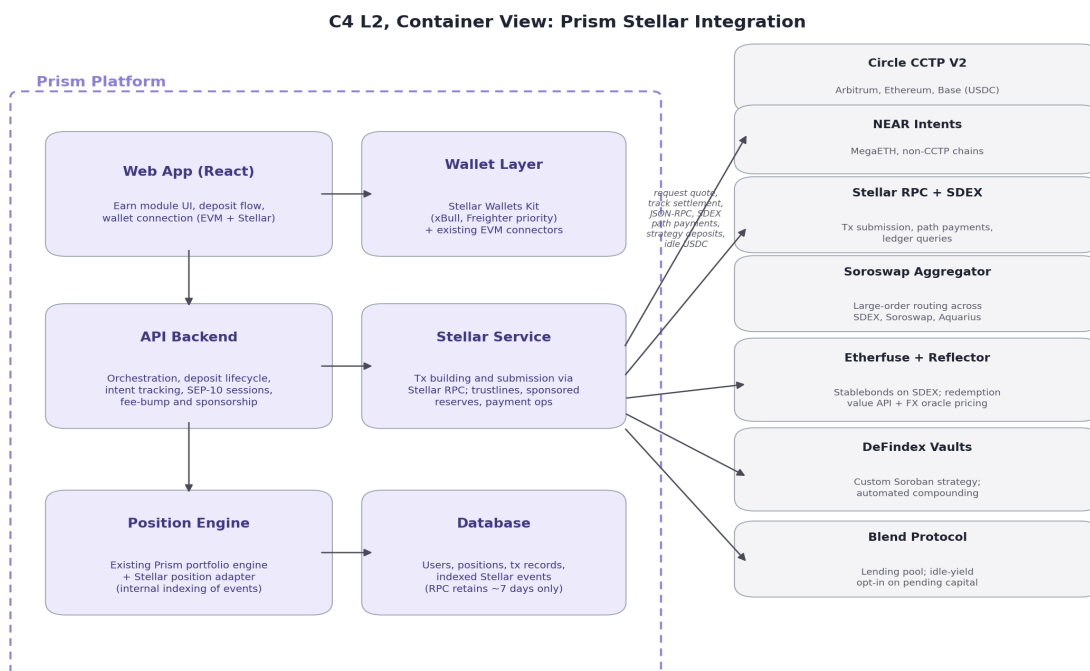


Figure 2: Container view. Blue containers are Prism components (existing or extended); gray boxes are external systems.

MegaETH, Prism's primary chain with 6,000+ users, is not in Circle's CCTP domain coverage. NEAR Intents is required, not optional, for this user base. Route selection is backend logic; the user experience is identical on both paths.

4. What Is Already Built (POC)

A POC is deployed at stellar.prismfi.cc/earn, built before requesting any funding. It contains two things: a complete interactive preview of the earn module, and a functional Stellar wallet integration running on testnet.

- **Product preview (interface).** The full user journey is designed and deployed: the deposit entry point, the three-step flow (deposit USDC, route to Stellar, allocate to stablebonds), the market selection screen (e.g., USTRY as first market, multi-currency expansion), and the position display inside Prism's UI. The product design is final; what remains is wiring it to live flows.
- **Functional Stellar wallet layer (testnet).** This part runs today: Stellar Wallets Kit connection, testnet account creation, on-ledger balance reads, and ownership-proof signing. It is the first Stellar integration code in the Prism codebase, and it validates the wallet layer described in section 5.1.

The Build Award funds the implementation of the flows behind this interface: the cross-chain deposit routing, the yield allocation and redemption lifecycle, durable position indexing, and production monitoring, as specified in section 5. No tranche includes research or system design: the architecture in this document is final, and part of the integration surface is already running.

5. Integration Design

5.1 Account model and EVM user onboarding

Two user paths are supported. **Stellar-native users** connect through Stellar Wallets Kit (Creit Tech), the single library covering all major Stellar wallets, with xBull and Freighter as priority targets. **EVM-native users**, the majority of Prism's base, never need to know a Stellar account exists: NEAR Chain Signatures derives a non-custodial Stellar account controlled by the user's EVM key, so Stellar execution happens behind the scenes without a new wallet or seed phrase. Prism covers the account's base reserve and asset trustlines

through sponsored reserves (BeginSponsoringFutureReserves / EndSponsoringFutureReserves, ChangeTrust) and submits operations through fee-bump transactions, so the user never holds or spends XLM.

Efficiency rationale: sponsored reserves and fee-bumps are native protocol features, zero custom contract code, and the standard pattern recommended for invisible onboarding. Stellar Wallets Kit replaces N wallet integrations with one dependency.

Session management. Stellar-native users authenticate through SEP-10 Web Authentication after wallet connection: the backend issues a ManageData challenge transaction, the user signs through Stellar Wallets Kit, and the backend returns a session JWT linking portfolio history and preferences to the Stellar identity across browser sessions. EVM-native users inherit session continuity from existing Prism auth; their derived Stellar account is resolved server-side.

Forward-compatibility: Protocol 27. Protocol 27 ("Zipper", testnet June 18 2026, mainnet vote July 8 2026) introduces first-class authentication delegation for smart accounts. The account layer is designed so smart-account delegation can be added alongside NEAR Chain Signatures as a V1.x option without changing the deposit or yield flows. The account layer stays flexible as the smart-account standard matures, while running on a production-ready NEAR rail today.

5.2 Bridge layer: cross-chain USDC routing

Deposits route into Stellar through two complementary rails, selected by the backend per source chain and asset. **Circle CCTP V2**, live on Stellar mainnet since May 2026, is the canonical route for native USDC from CCTP domains such as Arbitrum: 1:1 burn-and-mint with no wrapped assets, no solver liquidity caps, and a hooks mechanism designed for atomic post-mint actions on arrival. **NEAR Intents** covers chains outside CCTP coverage (e.g., MegaETH) and non-USDC source assets: the backend requests a quote, the user signs a single transaction on their EVM wallet, and the solver network settles USDC on the user's Stellar account, with settlement typically completing in seconds. Solver liquidity caps apply only to the intents route; the UX caps single transfers there and splits larger amounts into sequential intents. Failure handling on both routes: transfers are tracked to finality and unconfirmed transfers surface a retry path in the UI. Withdrawal back to EVM uses the same routing in reverse. The rail is live and launched with strong ecosystem backing; consumer-facing integrations are only beginning, and reaching the route still takes a multi-step journey. Prism's contribution is distribution: packaging it into one click and bringing an existing user base, TVL, and activity onto it.

Efficiency rationale: two production ecosystem rails, zero custom bridge contracts to build, run, or operate. CCTP V2 is Circle's canonical infrastructure; NEAR Intents extends coverage to 20+ chains and non-USDC assets. Route selection is backend logic, not new infrastructure.

5.3 RWA yield layer: stablebonds via SDEX and aggregated routing

Yield positions are denominated in Etherfuse stablebonds, tokenized sovereign bond exposure issued on Stellar as standard Stellar assets. USTRY (US Treasury) is the first market; the structural advantage is the multi-currency suite behind it (CETES in MXN, TESOURO in BRL, EUROB in EUR), sovereign yield exposure that is not available to EVM users today and is the core reason Stellar is the rail for this module. Acquisition runs through two paths selected by order size. Below a configurable threshold, the Stellar Service issues a PathPaymentStrictReceive directly on the **SDEX**, Stellar's native on-ledger exchange: atomic, gas-efficient, no custom contracts. Above the threshold, orders route through the **Soroswap Aggregator**, which splits across SDEX, Soroswap AMM pools, and Aquarius stableswap pools for best execution: the Router SDK simulates split distributions off-chain and the Aggregator contract executes atomically on-chain. The issuer confirms the deep liquidity venue for these assets is Stellar and that USTRY is purchasable today, permissionless, with no KYC on the secondary market. A protocol constraint shapes this: SDEX path payments are Classic operations while Soroswap and Aquarius routes use Soroban (InvokeHostFunction), and a single Stellar transaction cannot mix the two. Acquisition routes are therefore protocol-homogeneous,

the backend selecting one optimal route per order; the user never manages the distinction. Stablebonds are accreting-value tokens (balance stays constant, redemption value increases with accrued yield), so position pricing combines the issuer's redemption-value API with FX feeds from **Reflector**, Stellar's native oracle, for multi-currency display (off-chain; on-chain oracle use is covered in 5.4). Position accounting extends Prism's existing position engine with a Stellar adapter: relevant ledger events are read through Stellar RPC and indexed internally (RPC retains ~7 days of history, so durable history lives in Prism's database, consistent with SDF's recommended ingest-and-index pattern).

Efficiency rationale: small orders use SDEX path payments (native protocol feature), large orders use the Soroswap Aggregator (battle-tested, SCF-funded routing). No custom AMM or orderbook. Volume distributes across three liquidity venues, deepening ecosystem liquidity. Reflector and the issuer's pricing API replace any custom pricing infrastructure.

5.4 Strategy layer: custom DeFindex strategy

On top of direct stablebond positions, Prism ships a **custom Soroban strategy on DeFindex's vault framework**, aggregating stablebond exposure with automated compounding and plugging into DeFindex's revenue share framework. DeFindex is SCF-funded vault infrastructure: apps plug one API to offer vault products, and vaults route into Blend pools with reward swaps via Soroswap (Soroswap and DeFindex are both built by the same team, PaltaLabs, so the strategy and the routing layer share one integration relationship). It is the only custom Soroban surface in the project. The core deposit and yield path ships with zero custom contracts and does not depend on it. The strategy is open-sourced post-launch and hardened through SCF's standard tranche-3 security process. The team is already in contact with the PaltaLabs team. Revenue comes from a configurable performance fee on generated yield (a share of the yield, never of principal), distributed on-chain and split with DeFindex, with the user's displayed APY always shown net. The fee is set low by design: on consumer sovereign-bond yield, keeping net returns competitive matters more than fee extraction, so adoption comes first and the fee scales with the value a managed strategy adds.

Oracle risk containment. The V1 strategy is single-asset (USTRY) by design and accounts in bond units: the share price is bond per share, value accrual lives in the bond itself, and the contract reads no price feed. No feed, no share-price manipulation vector, and a smaller review surface. Multi-bond aggregation is the V1.x extension and introduces on-chain FX feeds (Reflector) only with standard hardening: time-averaged prices, staleness checks, and deviation bounds. In V1, Reflector runs in **shadow mode**: the backend reads its FX feeds for display and benchmarks them against external sources, measuring reliability, staleness, and deviation in production before any contract depends on the feed.

Efficiency rationale: the strategy is a thin contract on a battle-tested, SCF-funded framework, not standalone vault infrastructure, and its V1 design is oracle-free by construction: less attack surface, less review surface, faster and safer Tranche 3. One integration inherits the composability stack (DeFindex, Blend, Soroswap) that SDF documents as the recommended building blocks.

5.5 Idle yield layer: Blend Protocol

Capital a user holds deliberately unallocated, between strategies or parked while deciding, can be put to work on **Blend**, an SCF-funded lending protocol on Stellar. This is an opt-in, not an automatic sweep: the fast deposit path is too short-lived to benefit, so Blend targets pending-allocation balances where the holding period is long enough to matter, and the user chooses to enable it. Supply and withdraw run through Blend's public interface (no custom contract) and are instant as long as the pool has liquidity; the integration plugs into a conservative, well-backstopped pool with a reliable oracle, and gates the opt-in past a high utilization threshold so a pending allocation is never blocked. Lending carries a residual risk (bad-debt socialization, standard to all lending markets) which is surfaced to the user at opt-in. Revenue on this layer uses Blend's Fee Vault in capped-rate mode: the user's return is prioritized up to a target rate and Prism keeps only the surplus, on-chain and with no custom billing. Blend usage here is distinct from the contract-managed Blend exposure inside the DeFindex strategy (5.4); the position engine tracks the two separately.

Efficiency rationale: Blend is a battle-tested, SCF-funded lending protocol; the integration uses its public interface with no custom contracts. Routing activity into a central ecosystem protocol also deepens Prism's relationship with the Stellar ecosystem rather than only consuming it.

6. Technology Stack

Backend

- Prism's existing production backend (orchestration, deposit lifecycle, cross-chain transfer tracking), extended with a Stellar service.
- **Stellar RPC (managed endpoint)** for transaction building, simulation, submission, and ledger queries. Chosen over Horizon per current SDF guidance for new applications; no self-hosted Horizon to operate.
- Stellar SDK (JavaScript/TypeScript) for transaction construction: ChangeTrust, sponsorship operations, payments, fee-bumps.
- Internal indexer: Stellar events relevant to Prism positions ingested into the existing database.
- **Reflector** price and FX feeds for multi-currency position pricing; issuer redemption-value API for stablebond NAV.
- **DeFindex** vault framework for the custom strategy layer (Soroban contract on their battle-tested framework).

Frontend

- Prism's existing React application; the earn module and deposit flow extend the current UI.
- **Stellar Wallets Kit** for Stellar wallet connectivity (xBull, Freightier priority; Lobstr, Albedo, WalletConnect supported by the same module).

Development tooling

- **Scaffold Stellar** as the reference toolkit for any contract work and TypeScript client generation; environments.toml mirrors the tranche structure (staging, testnet, mainnet).
- **Stellar Lab** for endpoint exploration and POC iteration.

Infrastructure and monitoring

- Existing Prism cloud infrastructure; the Stellar service deploys alongside current backend services.
- Tranche 3 adds monitoring and alerting on the Stellar module: bridge settlement latency (alert past 10 min on CCTP, 5 min on intents); transfer failure rate (circuit breaker past 1% over an hour); trustline and sponsorship errors (auto-retry with backoff, alert if exhausted); orderbook depth pre-trade check with fallback venue; Blend pool utilization gate past 95% on the idle-yield opt-in; nightly position reconciliation of indexed versus on-ledger state.

7. Architecture Constraints

- **Non-custodial by design.** Prism never takes custody of user funds. Invisible Stellar accounts are derived through NEAR Chain Signatures and controlled by the user's EVM key; Prism sponsors reserves and fees but cannot move funds. Smart-account delegation (Protocol 27) is a forward-compatible option as the standard matures.
- **KYC stays in regulated perimeters.** Issuer KYC applies to primary issuance and direct redemption with the issuer; Prism's flow uses the permissionless SDEX secondary market and does not operate a KYC pipeline for this module.
- **Regulated issuance of RWAs.** Stablebonds are issued by a regulated issuer; Prism provides interface access to on-ledger secondary-market liquidity and does not itself issue or distribute securities under license.
- **Jurisdictional gating.** RWA yield access is geo-gated per the issuer's distribution coverage; availability differs by user jurisdiction.
- **Inbound flow screening.** Cross-chain deposits inherit screening on the settlement path of each route; flagged flows are rejected before allocation.

7.1 Threat Model (STRIDE)

Category	Threat	Surface	Mitigation
Spoofing	Impersonation of a derived Stellar account	Account (5.1)	Chain Signatures: deterministic EVM-key derivation, verifiable on-chain; SEP-10 for Stellar-native users
Tampering	Manipulation of bridged amount or route	Bridge (5.2)	CCTP: Circle attestation verified before mint; intents: settlement verified on-chain
Repudiation	User denies a transaction	Data (5.3, 6)	All ops signed on-chain (EVM and Stellar); indexer persists history independently of RPC
Info disclosure	Leak of positions or balances	Position engine	Public ledger by design; no private keys in DB; expiring JWT sessions
Denial of service	SDEX depth thin or Blend over-utilized	Yield (5.3, 5.5)	Aggregator fallback venues; Blend utilization check; circuit breaker with retry
Elevation of privilege	Unauthorized vault management	Strategy (5.4)	DeFindex roles: Vault Manager (multisig), Rebalancer (bot), Emergency Manager

8. Design-to-Deliverables Mapping

Each deliverable in the Build submission maps to a section of this document. Reviewers validating tranches can use this table as the verification index.

Tranche	Deliverable	Design section
T1 (MVP)	Production-grade EVM-to-Stellar deposit flow	5.2 Bridge layer
T1 (MVP)	Stellar wallet connectivity and SEP-10 sessions	5.1 Account model
T1 (MVP)	RWA yield position engine	5.3 Yield layer
T2 (Testnet)	End-to-end deposit and earn flow on testnet	5.1 + 5.2 + 5.3
T2 (Testnet)	Redemption, Soroswap routing, and Blend idle yield	5.2 + 5.3 + 5.5
T2 (Testnet)	Position history indexer and portfolio integration	5.3 + 6 Backend
T3 (Mainnet)	Mainnet launch of the Stellar earn module	All of section 5
T3 (Mainnet)	Custom DeFindex vault strategy (Soroban)	5.4 Strategy layer
T3 (Mainnet)	Launch hardening: withdrawal, monitoring, docs, testing	5.2 + 6 Infrastructure

Document owner: Prism engineering. Version 3.1, June 2026. Prepared for the Stellar Community Fund Build Award submission (Integration Track).